

Chapter 3

People You May Know

A temporal graph recommendation case study

PRIMARY REFERENCE

Study foundation

This chapter began as personal study notes based largely on Machine Learning System Design Interview by Ali Aminian and Alex Xu. The restructuring, clarifications, trade-off discussions, diagrams, and interpretations are the author's own.

Chapter at a glance

Design area	Current direction
Objective	Recommend non-connected users likely to form a meaningful future connection
Candidate generation	Bounded friends-of-friends plus complementary graph and cold-start sources
Baseline	Pointwise binary classifier or learning-to-rank model
Richer model	Temporal GNN for time-aware edge prediction
Pair scoring	Fast graph or dot-product narrowing followed by richer MLP scoring when justified
Serving	Precompute and cache; refresh lazily; optionally rerank with fresh lightweight features
Feedback	Treat repeated non-action and delayed connection attribution explicitly

1. Case map and minimum scope

Generate a plausible candidate pool, estimate future connection relevance, and return a short ranked list. Do not begin with all-pairs graph prediction.

People You May Know design map

Move from product objective and graph scale to candidate generation, temporal scoring, serving, and feedback.



Minimum-scope principle

Begin with graph-based candidate generation and a pointwise or learning-to-rank baseline. Add temporal node memory only when recency features cannot capture the required lift.

Figure 3.1 - People You May Know design map.

CONFIRMED INTERPRETATION

Prediction internally, ranking externally

The model estimates whether a future edge is likely, while the product shows only a short ordered list. Classification and ranking metrics therefore answer different questions.

OPEN DESIGN DISCUSSION

What is the minimum sufficient service?

Primary product outcome: _____

Candidate boundary: _____

Visible K: _____

Latency assumption: _____

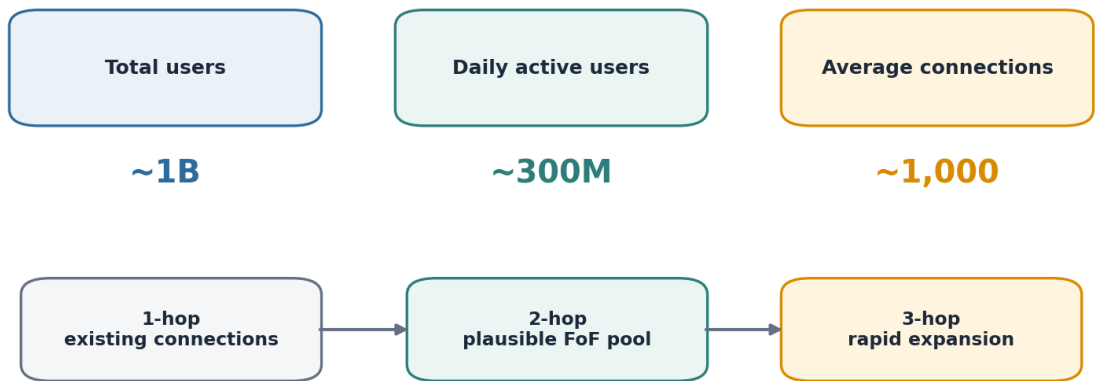
Explicitly out of scope: _____

2. Scale assumptions and graph constraints

The notes use large illustrative numbers to reason about storage and graph expansion. They should not be presented as verified platform statistics.

Scale changes the architecture

The numbers are capacity-planning assumptions from the notes, not verified facts about a specific platform.



Design consequence: exclude current connections, cap graph expansion, deduplicate candidates, and rank before expensive pair modeling.

Figure 3.2 - Capacity assumptions and candidate expansion.

Constraint	Design consequence	Reason
Existing 1-hop connections	Exclude from new-connection candidates	They are already connected
Large 2-hop pool	Cap, deduplicate, and rank candidates	Average degree can create a very large FoF set
3-hop expansion	Use selectively	Coverage grows quickly with substantial cost and noise
Inactive users	Avoid unnecessary proactive recomputation	Not every account needs a fresh list continuously

OPEN DESIGN DISCUSSION

Challenge the capacity assumptions

Revised total-user assumption: _____

Revised active-user assumption: _____

Revised average degree: _____

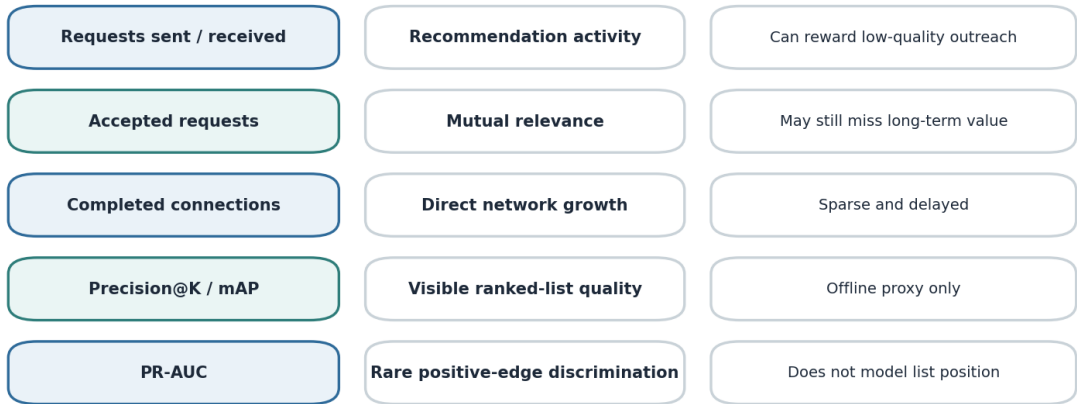
Architecture impact: _____

3. Product and evaluation metrics

Requests measure recommendation activity; accepted and completed connections better represent mutual value. Offline metrics should reflect both rare positives and the visible ranked list.

Metrics separate attraction from mutual value

A request sent is not the same outcome as an accepted or durable connection.



Starting recommendation: gate the model with top-K and rare-positive metrics offline, then optimize accepted or completed connections online with safety and spam guardrails.

Figure 3.3 - Metric families and limitations.

Metric family	Examples	Decision supported
Online activity	Requests sent and received	Do recommendations trigger outreach?
Mutual outcome	Accepted requests and completed connections	Did both users find the match relevant?
Longer-term product	Time to connection and network growth	Did the system create durable value?
Offline classification	PR-AUC, ROC-AUC, precision, recall	Can the model separate future edges?
Offline ranking	Precision@K and mAP	Is the visible list ordered well?

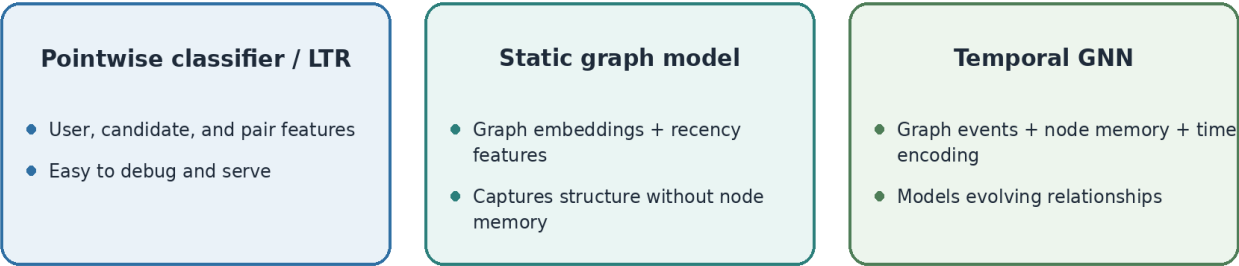
STARTING RECOMMENDATION
Optimize mutual outcomes, not request volume alone
Use Precision@K or mAP and PR-AUC offline, then evaluate accepted or completed connections online. Track spam, hide, block, and low-quality-request behavior as guardrails.

4. Baseline versus temporal graph modeling

A temporal GNN can represent evolving relationships, but it should earn its operational cost against simpler supervised and static graph baselines.

Baseline versus graph-aware models

The temporal GNN is the richer proposal, not the required starting point.



Promotion rule

Keep the simpler model unless event order and memory produce measurable lift that justifies state, training, and serving complexity.

Figure 3.4 - Model alternatives by complexity.

Model direction	Main benefit	Main cost
Pointwise classifier or LTR	Simple training, serving, and debugging	Requires manual graph and recency features
Static graph model + recency features	Captures topology without per-node temporal state	May lose event order and evolving memory
Temporal GNN	Models graph events, node memory, and time	Higher state, compute, and operational complexity

OPEN DESIGN DISCUSSION

Is temporal memory justified?

Expected lift over pointwise baseline: _____
Event order that matters: _____
Static-model alternative: _____
Complexity threshold for rejection: _____

5. Data, features, and social affinity

Represent users, graph structure, direct activity, recency, and exposure history. Standardize education and work text before treating it as categorical affinity.

Build time-aware social affinity from several signal families

Mutual-connection count alone may miss recency, engagement, and repeated non-response.

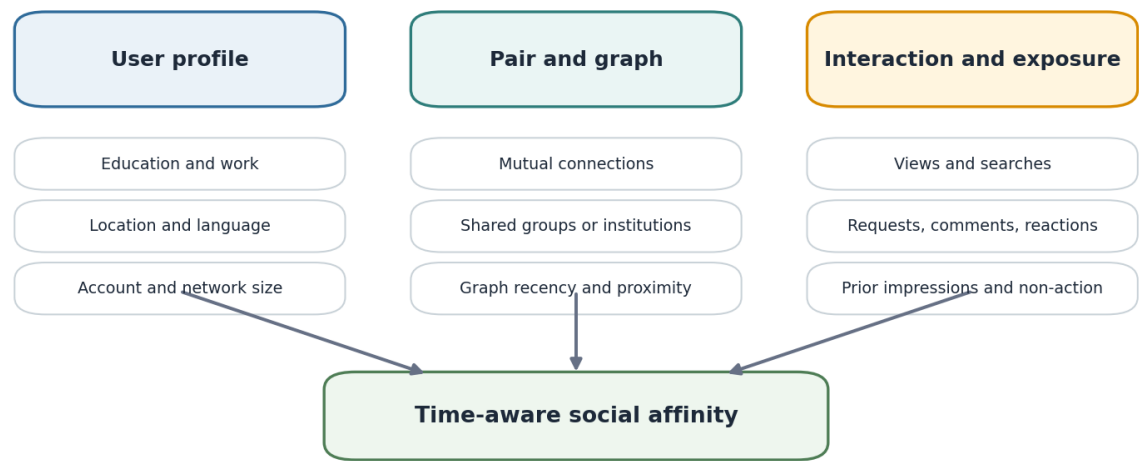


Figure 3.5 - Feature families that form time-aware social affinity.

Feature family	Examples	Important control
User profile	School, degree, major, work, location, language	Normalize equivalent text such as CS and Computer Science
Graph	Mutual connections, graph hops, shared institutions	Use only information available before scoring
Direct interaction	Profile views, searches, requests, comments, reactions	Preserve timestamp and direction
Exposure	Prior recommendation impressions and time since exposure	Distinguish never shown from repeatedly ignored

CONFIRMED INTERPRETATION

Recency alone is insufficient

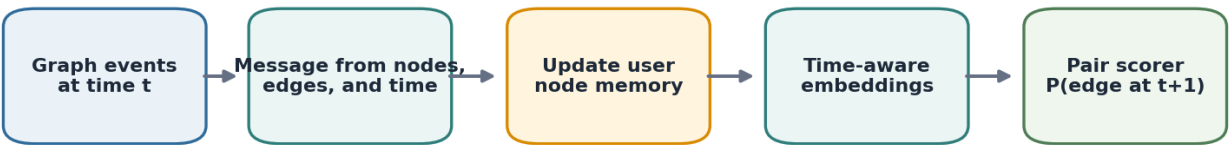
Social affinity may combine recent mutual connections, direct engagement, sustained older engagement, graph changes, and repeated opportunities without action.

6. Temporal GNN architecture and node memory

At each event time, build messages from users, edges, features, and time; update memory; create embeddings; and score a future pair.

Temporal GNN edge-prediction flow

Events update user memory, which produces time-aware embeddings for future-edge scoring.



Possible memory contents



LSTM or GRU components are examples from the notes; the exact recurrent mechanism remains an implementation choice.

Figure 3.6 - Temporal GNN edge-prediction flow.

Memory design question	Possible content	Trade-off
Event coverage	Connections, views, searches, requests, shared-network engagement	More event types increase complexity and noise
History length	Fixed window or time decay	Longer history costs more and may preserve stale behavior
Per-user memory	Compact recurrent state	Larger state may improve detail but harms serving cost
Inactive users	Decay, archive, or lazy reactivation	Aggressive cleanup may lose useful history

OPEN DESIGN DISCUSSION

What should node memory retain?

Event types: _____

History or decay rule: _____

Per-user state budget: _____

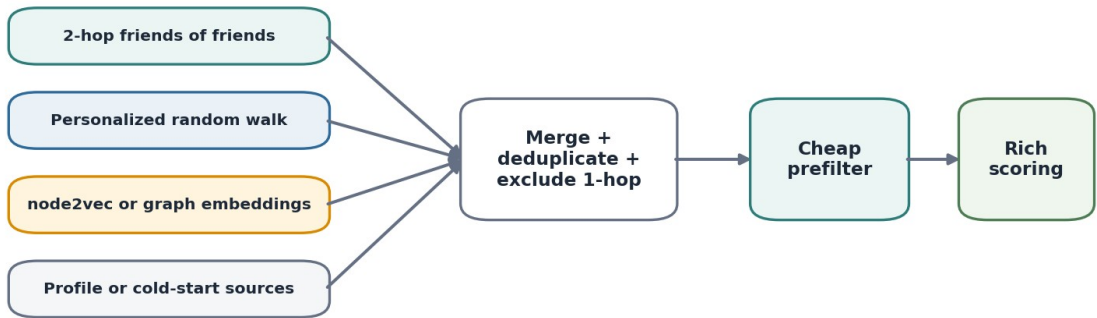
Inactive-user handling: _____

7. Candidate generation and graph alternatives

All-pairs prediction is infeasible. Candidate generation should use graph proximity and complementary sources before rich edge prediction.

Candidate generation should combine graph sources

Use inexpensive graph methods to produce a bounded pool before expensive scoring.



The notes contain a rough, unverified FoF observation. The safe design conclusion is only that FoF is a plausible high-value candidate source, not that a specific percentage is established.

Figure 3.7 - Candidate-source merge and staged scoring.

Candidate source	Best when	Main limitation
2-hop friends of friends	Graph structure is informative and mutual connections matter	May under-cover sparse or new users
Personalized random walk	Local graph proximity beyond raw hop count is useful	Can be costly or biased toward dense regions
node2vec / graph embedding	Reusable graph representations support retrieval	May become stale and lose event timing
Profile or contextual source	Cold-start users lack graph history	May be weaker than real social signals

SOURCE CAUTION

Do not publish the rough FoF percentage as verified fact

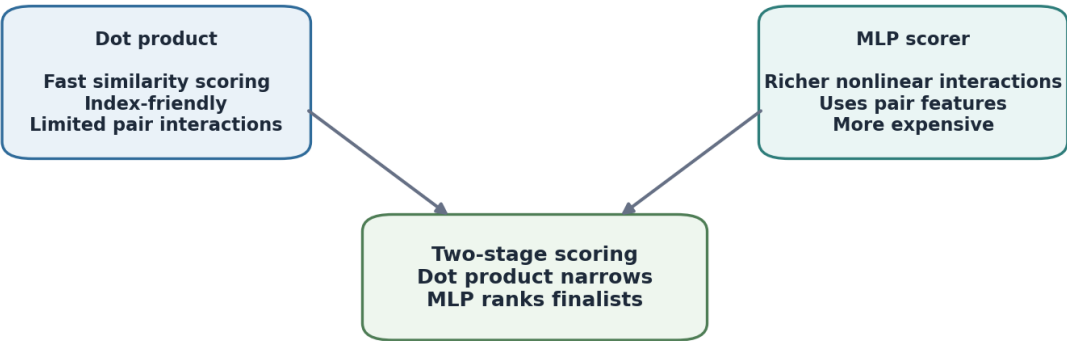
The original notes record an approximate external observation. This public chapter keeps only the design implication that friends-of-friends are a plausible high-value source.

8. Pair scoring: dot product, MLP, or both

Fast similarity scoring supports large candidate pools; richer pair modeling supports final ranking. They may be alternatives or sequential stages.

Dot product and MLP solve different scoring constraints

Use one or both depending on retrieval scale and pair-feature requirements.



Starting recommendation: use fast graph or embedding scores for candidate reduction, then use an MLP only where richer pair features provide measurable ranking lift.

Figure 3.8 - Pair-scoring alternatives.

Scorer	Best when	Main limitation
Dot product	Embedding retrieval and high-throughput narrowing	Limited interaction flexibility
MLP	Pair features and nonlinear interactions matter	More expensive and less index-friendly
Dot product then MLP	Scale and rich final scoring are both required	Two-stage pipeline and score alignment

STARTING RECOMMENDATION

Use rich scoring only on a bounded set

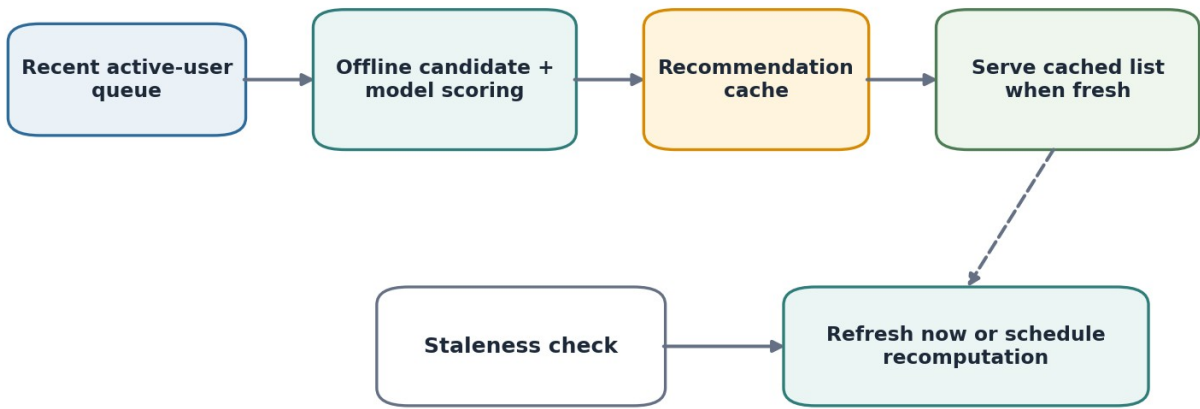
Generate and narrow candidates with graph rules, embeddings, or a lightweight scorer. Apply an MLP only to the smaller finalist set when pair-feature lift justifies the cost.

9. Serving, precomputation, and lazy refresh

The social graph changes more slowly than a content feed, so cached recommendations can reduce cost without recomputing after every event.

Precompute, cache, and refresh lazily

Avoid recomputing recommendations after every small graph event.



A lightweight ranker may prefilter before expensive scoring, rerank cached candidates with fresh features, or perform both roles.

Figure 3.9 - Cached serving with lazy refresh.

Serving choice	Benefit	Risk
Precompute for recently active users	Focuses compute on likely traffic	A returning inactive user may receive stale results
Lazy refresh on request	Avoids event-by-event recomputation	First stale request may need fallback behavior
Lightweight fresh reranker	Uses recent features without full recomputation	Adds online feature and model dependencies
Immediate graph-event refresh	Maximum freshness	Often unnecessary and expensive

CONFIRMED INTERPRETATION

Precompute plus lazy refresh is the preferred serving idea

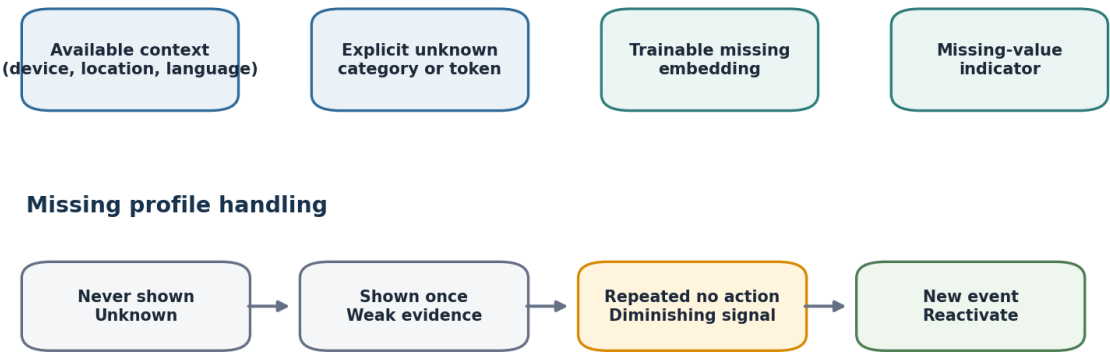
Serve the cached list when it is fresh. When stale, refresh immediately or schedule recomputation. A lightweight ranker may prefilter, rerank cached candidates, or do both.

10. Missing profiles, exposure, and delayed feedback

The service should distinguish unavailable profile fields from actual values and should treat repeated non-action differently from a single ignored impression.

Missing profiles and repeated exposure need explicit treatment

Missingness is information; non-action becomes stronger only after repeated opportunities.



New graph changes, profile views, searches, or shared-network engagement can restore a previously suppressed candidate.

Figure 3.10 - Missing-value options and exposure states.

Problem	Options	Decision principle
Missing profile fields	Available context, unknown token, trainable embedding, indicator	The options may be combined; model why the value is missing
Ignored recommendation	No penalty, decay, temporary suppression	One ignore is weak; repeated exposure is stronger evidence
Candidate restoration	New graph or engagement signals	Do not suppress permanently when the relationship changes
Delayed connection credit	Fixed window, decay, recent-touch, multi-touch, dynamic attribution	Use time and intervening events, not elapsed time alone

OPEN DESIGN DISCUSSION

How should ignored and delayed outcomes be attributed?

Exposure penalty: _____
Suppression duration: _____
Reactivation signal: _____
Label window: _____
Attribution method: _____

11. Negative-pair sampling and online evaluation

Negative-pair sampling is an added technical extension because future connections are rare and naive random negatives may be too easy.

Training negatives and online comparison

Negative-pair sampling is an added technical extension, clearly separated from the original notes.

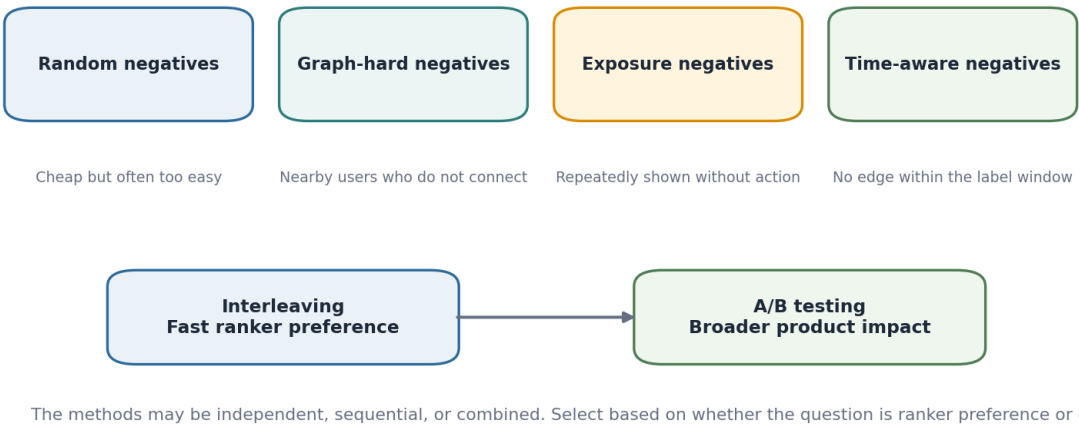


Figure 3.11 - Negative-pair and online-evaluation options.

Negative strategy	Benefit	Risk
Random users	Simple and scalable	Often trivially easy and unlike serving candidates
Graph-hard negatives	Matches plausible nearby users	May include future positives
Exposure negatives	Reflects repeated opportunity without action	Affected by ranking and exposure bias
Time-aware negatives	Respects a defined future-edge window	Window choice changes the label

Online method	Best use	Limitation
Interleaving	Fast preference comparison between rankers	Narrower than full product impact
A/B testing	Causal effect on connections and guardrails	Slower and requires traffic
Sequential or combined	Fast ranking signal followed by product validation	More experiment coordination

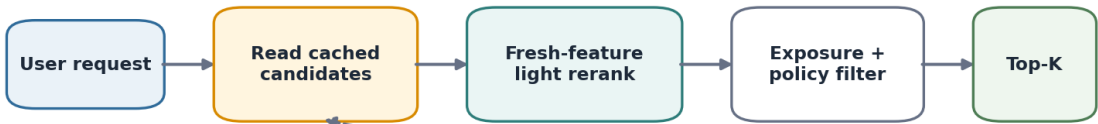
12. Reference architecture

One defensible system combines bounded graph candidates, supervised scoring, cache-aware serving, explicit exposure policy, and evidence-based temporal-model promotion.

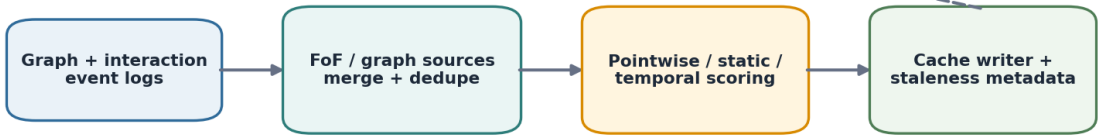
Reference architecture

One production-balanced design under the current assumptions.

ONLINE SERVING



OFFLINE GRAPH LEARNING AND REFRESH



Starting recommendation: graph-based bounded candidate generation, a simple supervised scorer first, cached recommendations with lazy refresh, and temporal GNN promotion only after measurable incremental value.

Figure 3.12 - Production-balanced reference architecture.

ONE REASONABLE DESIGN

Current reference architecture

Build candidates from bounded friends-of-friends and complementary graph or profile sources, merge and exclude current connections, score with a pointwise or static model first, write a cached list with staleness metadata, and rerank lightly at request time. Promote a temporal GNN only when event memory produces meaningful lift.

OPEN DESIGN DISCUSSION

What would change this architecture?

Candidate-pool threshold: _____

Freshness requirement: _____

Temporal-model lift requirement: _____

Per-user state budget: _____

Simpler acceptable design: _____

13. Decision summary

The starting choices remain deliberately reversible. Each switch condition identifies the evidence required to add or remove complexity.

Design area	Starting choice	Switch when
Product outcome	Accepted or completed connections	Request activity is proven to be the right business target
Candidate source	Bounded 2-hop FoF plus complementary sources	Coverage or quality requires broader graph traversal
Baseline	Pointwise classifier or LTR	A static graph model clearly improves quality
Richer model	Temporal GNN as a promoted experiment	Temporal lift does not justify state and serving cost
Affinity	Graph, recency, engagement, and exposure features	A smaller feature set performs equivalently
Pair scoring	Cheap narrowing then MLP finalists	One scorer satisfies both scale and quality
Serving	Precompute, cache, and lazy refresh	Freshness loss harms outcomes
Missing data	Explicit missingness plus available context	A simpler treatment performs equivalently
Ignored items	Gradual penalty and temporary suppression	Non-action is shown to be non-informative
Evaluation	Top-K + rare-positive metrics, then online test	Offline metrics fail to predict product impact

INTERVIEW-READY ANSWER

Trace every component to scale, freshness, or relationship quality

I would begin with bounded friends-of-friends and complementary candidate sources, then use a pointwise or learning-to-rank baseline with graph, profile, recency, and exposure features. I would cache lists and refresh lazily, use a lightweight fresh reranker when needed, and promote a temporal GNN only after measurable lift over static recency features. I would evaluate Precision@K or mAP and PR-AUC offline, then accepted or completed connections with safety guardrails online.

Attribution

This project is independent study work and is not affiliated with the book authors or publisher. It is not a substitute for the original book. Public versions use original wording and original diagrams and exclude handwritten source scans.